

Documentação do `src/lib/formatter.js`

Objetivo

O arquivo `src/lib/formatter.js` centraliza funções utilitárias para:

- limpar valores
- aplicar máscaras em documentos e campos
- formatar valores monetários em real
- preparar texto para inputs com máscara dinâmica
- validar CPF, CNPJ, telefone e CEP

Ele serve para evitar lógica duplicada no front e padronizar o comportamento dos campos do sistema.

Caminho

`src/lib/formatter.js`

Visão geral das funções

Helpers

Funções base para limpeza de strings.

- `onlyDigits`
- `onlyMoneyDigits`
- `removeMask`

Formatação de documentos

- `formatCPF`
- `formatCNPJ`
- `formatCpfCnpj`

Formatação de contato e endereço

- `formatPhone`
- `formatCEP`

Formatação monetária

- `formatBRL`
- `parseBRLToNumber`
- `parseBRLToCents`

Formatação para inputs

- `formatCPFInput`
- `formatCNPJInput`
- `formatCpfCnpjInput`
- `formatPhoneInput`
- `formatCEPInput`
- `formatBRLInput`

Validações

- `isValidCPF`
 - `isValidCNPJ`
 - `isValidPhone`
 - `isValidCEP`
-

1) Helpers

`onlyDigits(value = "")`

Objetivo

Remove qualquer caractere que não seja número.

Parâmetros

- `value`: string, número ou valor convertível para string

Retorno

- string contendo apenas dígitos

Exemplo

```
onlyDigits("123.456-78");  
// "12345678"
```

Uso comum

Ideal para limpar CPF, CNPJ, telefone, CEP e outros campos numéricos antes de formatar ou validar.

onlyMoneyDigits(value = "")

Objetivo

Remove tudo que não for número de um valor monetário.

Retorno

- string contendo apenas dígitos

Exemplo

```
onlyMoneyDigits("R$ 1.234,56");  
// "123456"
```

Observação

Hoje ela faz exatamente a mesma coisa que `onlyDigits`. Então, sendo bem sincero, ela existe mais por clareza semântica do que por necessidade técnica. Isso não é errado, mas é redundante.

removeMask(value = "")

Objetivo

Remove máscara de um valor, deixando apenas os números.

Retorno

- string sem caracteres especiais

Exemplo

```
removeMask("(32) 99999-8888");  
// "32999998888"
```

Observação

Na prática ela também faz a mesma coisa que `onlyDigits`. Ou seja: funcionalmente está certa, mas há repetição no arquivo.

2) Formatação de CPF

`formatCPF(value = "")`

Objetivo

Aplica máscara progressiva de CPF.

Regra de formatação

Formato final:

000.000.000-00

Comportamento

A máscara é aplicada conforme o usuário digita.

Exemplo de progressão

```
formatCPF("1"); // "1"  
formatCPF("123"); // "123"  
formatCPF("1234"); // "123.4"  
formatCPF("123456"); // "123.456"  
formatCPF("12345678901"); // "123.456.789-01"
```

Limite

- máximo de 11 dígitos
-

3) Formatação de CNPJ

formatCNPJ(value = "")

Objetivo

Aplica máscara progressiva de CNPJ.

Regra de formatação

Formato final:

00.000.000/0000-00

Exemplo de progressão

formatCNPJ("12"); // "12"

formatCNPJ("123"); // "12.3"

formatCNPJ("12345678"); // "12.345.678"

formatCNPJ("12345678000199"); // "12.345.678/0001-99"

Limite

- máximo de 14 dígitos
-

4) Formatação automática de CPF ou CNPJ

formatCpfCnpj(value = "")

Objetivo

Detecta automaticamente se o valor deve ser tratado como CPF ou CNPJ.

Regra

- até 11 dígitos: aplica CPF
- acima de 11 dígitos: aplica CNPJ

Exemplo

```
formatCpfCnpj("12345678901");  
// "123.456.789-01"
```

```
formatCpfCnpj("12345678000199");  
// "12.345.678/0001-99"
```

Uso recomendado

Excelente para campos genéricos de “CPF/CNPJ” no cadastro de cliente, proprietário, empresa, etc.

5) Formatação de telefone

formatPhone(value = "")

Objetivo

Aplica máscara para telefone brasileiro.

Regras

- 10 dígitos: telefone fixo
- 11 dígitos: celular

Formatos finais

(32) 3333-4444
(32) 99999-8888

Exemplo de progressão

```
formatPhone("3"); // "(3"  
formatPhone("32"); // "(32"  
formatPhone("329"); // "(32) 9"  
formatPhone("3233334444"); // "(32) 3333-4444"  
formatPhone("32999998888"); // "(32) 99999-8888"
```

Limite

- máximo de 11 dígitos

Observação importante

Essa função valida só o tamanho, não checa se o DDD existe ou se o número faz sentido real. Então ela resolve bem máscara e validação básica, mas não valida telefone “de verdade”.

6) Formatação de CEP

formatCEP(value = "")

Objetivo

Aplica máscara de CEP.

Formato final

00000-000

Exemplo

```
formatCEP("36309028");  
// "36309-028"
```

Limite

- máximo de 8 dígitos
-

7) Formatação monetária em BRL

formatBRL(value = "")

Objetivo

Converte entrada numérica em valor monetário formatado em real.

Regra

A função interpreta os dígitos como centavos.

Exemplos

```
formatBRL("1"); // "R$ 0,01"
```

```
formatBRL("12"); // "R$ 0,12"  
formatBRL("123"); // "R$ 1,23"  
formatBRL("1234"); // "R$ 12,34"
```

Comportamento especial

Se não houver dígitos:

```
formatBRL(""); // "R$ 0,00"
```

Observação

Esse tipo de máscara é ótimo para input de preço, comissão, valor de imóvel, aluguel, taxa, etc. É o padrão mais confortável para o usuário brasileiro.

parseBRLToNumber(value = "")

Objetivo

Converte uma string formatada em BRL para número JavaScript.

Exemplo

```
parseBRLToNumber("R$ 1.234,56");  
// 1234.56
```

Retorno

- número decimal JS
- retorna 0 se não conseguir converter

Uso recomendado

Bom para salvar no banco como `numeric`, `decimal` ou usar em cálculos no front.

parseBRLToCents(value = "")

Objetivo

Extraí apenas os dígitos de um valor monetário formatado.

Exemplo


```
parseBRLToCents("R$ 1.234,56");  
// "123456"
```

Uso recomendado

Bom quando você quer trabalhar com centavos inteiros, que muitas vezes é a forma mais segura para evitar bug de ponto flutuante.

8) Formataadores para uso em input

Essas funções existem para facilitar o uso direto em `onChange`.

Todas aceitam:

- uma `string`
 - ou um `event` com `event.target.value`
-

`formatCPFInput(eventOrValue)`

Aplica `formatCPF` em string ou evento.

Exemplo

```
onChange={(e) => setCpf(formatCPFInput(e))}
```

`formatCNPJInput(eventOrValue)`

Aplica `formatCNPJ`.

`formatCpfCnpjInput(eventOrValue)`

Aplica detecção automática entre CPF e CNPJ.

`formatPhoneInput(eventOrValue)`

Aplica máscara de telefone.

formatCEPInput(eventOrValue)

Aplica máscara de CEP.

formatBRLInput(eventOrValue)

Aplica máscara de dinheiro em real.

9) Validações básicas

isValidCPF(value = "")

Objetivo

Valida se o CPF é estruturalmente válido.

Regras aplicadas

- remove máscara
- exige 11 dígitos
- rejeita sequências repetidas como:
 - 11111111111
 - 00000000000
- calcula os dois dígitos verificadores

Exemplo

```
isValidCPF("529.982.247-25");  
// true
```

Observação

Essa validação é boa. Não é só máscara, ela realmente verifica dígito verificador.

isValidCNPJ(value = "")

Objetivo

Valida se o CNPJ é estruturalmente válido.

Regras aplicadas

- remove máscara
- exige 14 dígitos
- rejeita sequências repetidas
- calcula os dois dígitos verificadores com pesos oficiais

Exemplo

```
isValidCNPJ("11.222.333/0001-81");  
// true ou false conforme o número
```

Observação

Também é uma validação real de dígito verificador, não só conferência de tamanho.

isValidPhone(value = "")

Objetivo

Validação básica de telefone.

Regra

Aceita:

- 10 dígitos
- 11 dígitos

Exemplo

```
isValidPhone("(32) 99999-8888");  
// true
```

Limitação

Essa validação é superficial. Ela não verifica:

- DDD válido
- nono dígito obrigatório por região

- números impossíveis

Serve para front simples. Para algo mais sério, teria que endurecer isso.

isValidCEP(value = "")

Objetivo

Validação básica de CEP.

Regra

- precisa ter exatamente 8 dígitos

Exemplo

```
isValidCEP("36309-028");  
// true
```

Limitação

Não consulta existência real do CEP. Só confere estrutura.

Exemplos de uso no front

Input de CPF

```
const [cpf, setCpf] = useState("");  
  
<Input  
  value={cpf}  
  onChange={(e) => setCpf(formatCPFInput(e))}  
  placeholder="000.000.000-00"  
>
```

Input de telefone

```
const [telefone, setTelefone] = useState("");  
  
<Input  
  value={telefone}
```

```
onChange={(e) => setTelefone(formatPhoneInput(e))}  
placeholder="(00) 00000-0000"  
</>
```

Input de valor em reais

```
const [preco, setPreco] = useState("R$ 0,00");
```

```
<Input  
  value={preco}  
  onChange={(e) => setPreco(formatBRLInput(e))}  
  placeholder="R$ 0,00"  
</>
```

Antes de salvar no banco

```
const payload = {  
  cpf: removeMask(cpf),  
  telefone: removeMask(telefone),  
  cep: removeMask(cep),  
  preco: parseBRLToNumber(preco),  
};
```

Boas práticas de uso

Use máscara no input, mas salve limpo

Minha recomendação é firme aqui: no banco, salve sem máscara sempre que fizer sentido.

Exemplo:

- CPF: 12345678901
- telefone: 32999998888
- CEP: 36309028

Isso evita dor de cabeça em busca, comparação, indexação e integrações.

Valor monetário: prefira padrão consistente

Escolha uma estratégia e siga nela:

- salvar como decimal em reais, usando `parseBRLToNumber`
ou
- salvar em centavos inteiros, usando `parseBRLToCents`

Misturar as duas no sistema é convite para caos parcelado em 12 vezes.